ROSACE

Version 0.0.2

Table of Contents

Contents:

Introduction 4

Get started 6

How to Contribute 8

Basic principles 9

Step 1: session initialization 10

Step 2: Scheduler 11

Step 3: Sequencer 12

Step 4: Data Reduction 13

Step 5: Display, share 14

Step 6: Archive 15

Step 7: Closing the session 16

Observing file format 18

Welcome to the ROSACE project

ROSACE is the acronym for **Robotic Observations and Spectrocopic Astronomy, a Collaborative Experience**

The ROSACE project aims to develop a network of amateur and professional observatories to run Robotic Spectroscopic astronomical observations. All together, we offer a new tool to the Science to better understand our Universe.

The project is based on Open Source software tools only.

Robotic observation means that **no human action** is required. From opening the dome (or shelter) to processing the data and sharing the result, all is automated.

The project is developed by the community: any contribution is welcome. Our intention is to propose a system that can be adapted from smallest to biggest telescopes.

Introduction

Rosace proposes to create your own robotic observatory for Astronomical Spectro observations.

The name of the project is ROSACE, for **Robotic Observations and Spectrocopic Astronomy, a Collaborative Experience**

Our intention is that many observers, with different instruments and for various scientific programs can use this system. The final goal is that we can produce all together valuable data for the science, for a better understanding of the Universe we're living in.

We do think that Spectroscopic observations **cannot be fully standard** . The observation sequence will depend on your own setup, and the data reduction will vary from one scientific goal to the other. You will probably have to 'put your hands on', and write some code to adapt the system to your own need. We want to make these adaptations as simple as possible.

We also consider that the system must be able to manage several science programs ; none can use a telescope at 100% of the time. A science program can have its own observing sequence and own data reduction process. in ROSACE, each observation must be part of a given scientific program.

One key of a robotic observatory is that it really runs fully automatically. If each observation requires 2 minutes of human brain, it will quickly be overloaded - because every night can produce dozens of observations. Then a full automatic process is required. The only human part can be the validation, to make sure that all went well - the system must give all the tools to do it very fast.

To make this system accessible for everyone, we've organized the project with independant plugins, with few general rules, and the code is intended to be easy to read and maintain. Managing several small problems is better than a big one.

To help you starting with ROSACE, we propose this documentation, and a Get Started page . You can start with simulators, no need for a real instrument in a first step.

We work over the long term. People will change, instruments will change, new science opportunities will arise. The platform must be able to adapt to all these changes. To make this, we do our best to be clear on what is stable over the time (for instance the Observing File format), and what can evolve (the instruments, the observing sequences, and so on).

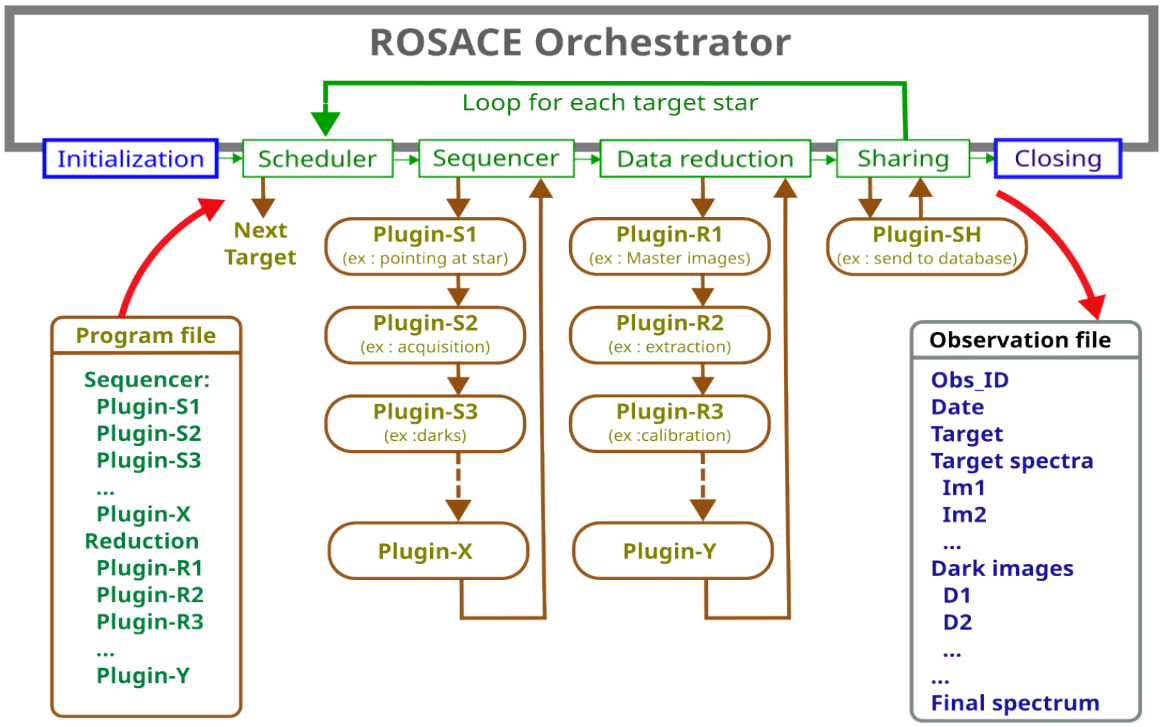
We think that continous improvement is the best way to move forward - one small step everyday.

The core of the system is an Orchestrator. It manages the observations from initialization of the observatory to its closure, and runs the observing and data reduction sequences for each target

stars. The observing sequence is made of plugins, that you can adapt to your own needs. And the same for the data reduction sequence.

Before starting the observing process, you must make sure that the observing conditions (weather) are good enough - this is out of the scope of ROSACE.

The Orchestrator runs different steps, one after the other. This is described in this figure:



It has a simple behavior: it starts the session by an INIT step (for instance to connect the hardware and cooling down the cameras). Then, it loops for each target the scheduler (to define what is the next target), the sequencer (for data acquisition), the data reduction and the sharing (to send the data to the right place, like a database). At the end of the session, it closes it (disconnect the hardware, and so on).

Each step for the sequencer and data reduction is made of plugins that you can adapt to your own needs.

The list of plugins that must be used for a given observation are descrived in a Program file (.yaml format). This means the each observation can have it own sequence (for data acquisiton, for data reduction, and for sharing).The plugins are dynamically loaded.

The Orchestrator creates an Observation file at the beginning of the observation. This file will go through all the steps (plugins), and each plugin will enrich it. At the end of the observation, this Observation file is the memory of the observation. All necessary data are recorded there: date, target, scientific program, raw images, data processing steps, and at the end the processed spectrum.

This Observation file is the only information that is known by each plugin. This makes all plugins independant from each other - and this is why it is easy to make or adapt your own plugins.

You'll find more details of each step in the following pages:

- Step 1: initialization (once at the beginning of the session)
- Step 2: Shelduler (what is the next target to observe)
- Step 3: Sequencer (run the actual observation)
- Step 4: Data reduction (to get a scientific result)
- Step 5: Display, validate and share the results
- Step 6: Archiving the results
- Step 7: Closing the session (once at the end of the session)

We do think that the success of this project depends on few elements :

- The community (developers and users) must be large enough ; the system must not depend on one or few guys.
- A clear and up-to-date documentation is available.
- We propose a 'get started' path, to jump quiclky and easily into the project.
- The team is very open to newcomers - you're welcome!

Get started

(this part is to be completed when the project will have enough maturity).

We propose here a step by step path to start with ROSACE.

1. The first step is to use a Docker container, with a demo version of ROSACE. It will allow you to test the system in a basic way, and quiclky understand the main features : start and stop a session, define targets to observe, look at the simulated images and the observations file, check the log, change the configuration files, and so on.

2. The next step is to install the application on your computer to test it in real conditions. We recommend to start with the INDI simulator devices (camera, mount). This can be managed by Kstars software. You must install few other applications for a working setup (Kstars, PHD2 guiding)... You'll be able to work with ROSACE in close to real conditions.

3. The final step is to connect your own instruments to the applications. You'll probably have to adapt few plugins, and configuration files - this will allow you to start running real observations!

... to be completed (Aug. 2026)

How to Contribute

Welcome to the ROSACE project .

The project is developed by the community, and we need you!

Here are the ways to help developing this tool:

- Install your own observatory and give us any user feedback
- If you’re a developper, contact us and tell you where you can help (there is so much to do!)
- Write and/or translate the documentation (the plan is to get only an english version, but we never know)

Basic principles

Welcome to the ROSACE project .

The project is based on few principles:

- An observation is a consistent set of images and data, described in an Observation File (yaml format).
- All files of one observing night are saved in a single folder.
- All images and spectra are in FTIS format (which is THE format for astronomy files).
- There is a single configuration file (in yaml format) for the whole application.
- There is also a hardware configuration file for each setup (telescope, mount, spectroscope, cameras...)
- We apply the TDD (Tests Driven Development) method. Each module or plugin has a series of tests to ensure there is no regression over the time.
- We propose a User Interface (UI) for the orchestrator, but the system can be used with no UI (which is the common way).
- All targets stars to be observed are stored in a local database (SQLite), to be able to observe even without internet connexion.
- We use only Open Source tools:
  - Linux [1] .
  - Python (3.10 or above) as much as possible.
  - Git for revision control.
  - Github repo to share the code.
  - Rest API interfaces (to split front-end and back-end).
  - User interfaces in (TypeScript - X4 framework). Works with any web browser.
  - INDI server and devices (INDI is for Linux what ASCOM is for Windows).
  - Sphinx for the documentation.

    - Few standard Python libraries :
        - Astropy, for all Astronomy related stuff.
        - FastAPI for the Rest API interfaces.
        - SQLite database for targets and observations records.
        - Logging for the loggers (all actions are recorded).

- The dome or shelter can open and close whatever is the telescope position [2] .
- Each module can be installed locally or remotely (client-server architecture).
- We can use Kstars for monitoring the actions (very useful for debugging).
- We love the concept of continuous improvement ; a small step every day.

Footnotes

[1]
We know that most of people are using Windows. The door is not closed. Since we use Python (which is not platform dependant) and ASCOM (under Windows) platform offers the same kind of feature than INDI (under Linux), this can probably be adapted in close future - despite Windows is not Open Source.

[2]
This is a simple and efficient way to split functions and protect the instrument. But we keep in mind that this condition is not made by lots of observatories (most of the rolling shelter ones), and there is no strong stopper to this.

Step 1: session initialization

The step is part of the ROSACE project

This step runs only once at the beginning of the session. It connects all the servers and devices, and starts few operations, like cooling down the cameras (if any).

A session_ID est created for each session, and stored in the sessions database.

All images and data of a given session are stored in a single folder on the disk. The folder name is based on the date and time. The folder is defined for a time range going from noon to noon the following day (to make sure that all data of a night are in a single folder).

This means that if several sessions are started during a same night (for instance if the weather is temporarly bad), they will all use the same folder.

Step 2: Scheduler

The step is part of the ROSACE project

Getting an observatory that is able to work automatically is good. But then, as soon as it is operationnal, the key question is: ‘what is my next target’.

This is the mission of the Step 2 to reply this question, and define at any time what is the next target to observe.

The strategy can be very simple - for example, you can set a list of targets in a text file.

But is can be also very rich if you have many observatories working together, managing multiple scientific programs. In such cases, the question of the next target can be tricky. This module will adapt to any cases - starting by the simple one, of course.

Step 3: Sequencer

The module is part of the ROSACE project

This module is the core of the Robotic observation. It runs all the steps of an observation sequence.

Each observation runs a given observing sequence. You can write your own sequence, to match your scientific need. The sequence is made of plugins ; you can use existing plugins or write your owns (in Python). We've blank plugins to help you creating your own.

Here are the main (and required) principles for this module:

1. Each observation uses a specific sequence (mainly linked to the scientific goal of the observation).
2. A sequence is a single list of basic steps. No loops, no conditions, the system is very simple.
3. A single structure (Observation object) is used to give each step all the data it needs (target star, exposure duration, and so on).
4. This structure can be append by each step.
5. This structure is also used at the end of the process to write the "Observing file", needed for the data reduction.
6. We use a common logging system ; each step can log any data you want.
7. If an error occurs during a step, an exception is raised, and the observation is stopped. Then a new observation is started, from the scheduler.

Step 4: Data Reduction

The step is part of the ROSACE project

Once the observing data is recorded, and the observation is completed, we must process it, to get the scientific result, corrected from any instrumental effect.

This is the job of this Step 4.

Of course, the data reduction process depends on each scientific program.

In a first time, we do this reduction step thanks to SpecINTI software. Then, we'll develop our own process - fully Open Source, in Python.

The data reduction process is a list of reduction operations (like master dark, spectrum extraction or wavelength calibration). ROSACE will propose a list of such operations, but you can write your owns, to adapt to your case.

Like for the Sequencer, this is the Observation file that moves the whole data of the observation through all the data reduction steps.

Step 5: Display, share

The step is part of the ROSACE project

At the end of the observing process, we must share our results and close the observation.

This step does it, with some important parts:

1. Validate the data by the observer (we never know, something may have happened).
2. Send the data to the relevant repository. Depending on the science case, it can be a database, as simple mail, or any other way.
3. Archive the data (raw data and result), and fullfill the local database, to make any future search fast and easy.

In a first time, we can make it manually, but as soon as the production increases, it must become automatic.

Step 6: Archive

The module is part of the ROSACE project

A reprendre

1. The data must be properly archived for long term usage. We keep all the raw data ; maybe we'll have better data reduction tools to improve the scientific results. Also, it is important to go back to raw data for any question.

Step 7: Closing the session

The step is part of the ROSACE project

At the end of the session, there are few operations to run, to close the observatory. There are mostly a symmetry from the INIT step: stopping the cooling of cameras (if any), disconnect the devices, and so on.

Observing file format

Each observation is fully defined by its Observing File. This file is in yaml format (human readable, easy to edit).

Here is the standard format for the Observing File (very temporary - to be completed):

```yaml
#Observing file format
#=====================
---
name: Vega
RA: "18h36m56.33635s"
DEC: "+38d47m01.28021975s"
nb: 3
exptime: 5
x1: 100
y1: 250
x2: 1500
y2: 1400
seq: Be_UVEX
reduction: Be_UVEX
```

Please send any comment or question to François Cochard (francois . cochard @ shelyak . com)